Faculty Profile

Dr. Piyush Bagla is a dedicated academician and researcher with a Ph.D. degree in Computer Science and Engineering, specializing in Machine Learning and Natural Language Processing from Dr. B.R. Ambedkar National Institute of Technology (NIT), Jalandhar, India. Dr. Bagla believes that effective teaching goes beyond theoretical instruction involves connecting concepts to real-world applications that resonate with students. For more detail, kindly visit the link below.

Profile Link - https://www.upes.ac.in/faculty/school-of-computer-science/dr-piyush-bagla

Object Oriented Programming: History and Evolution

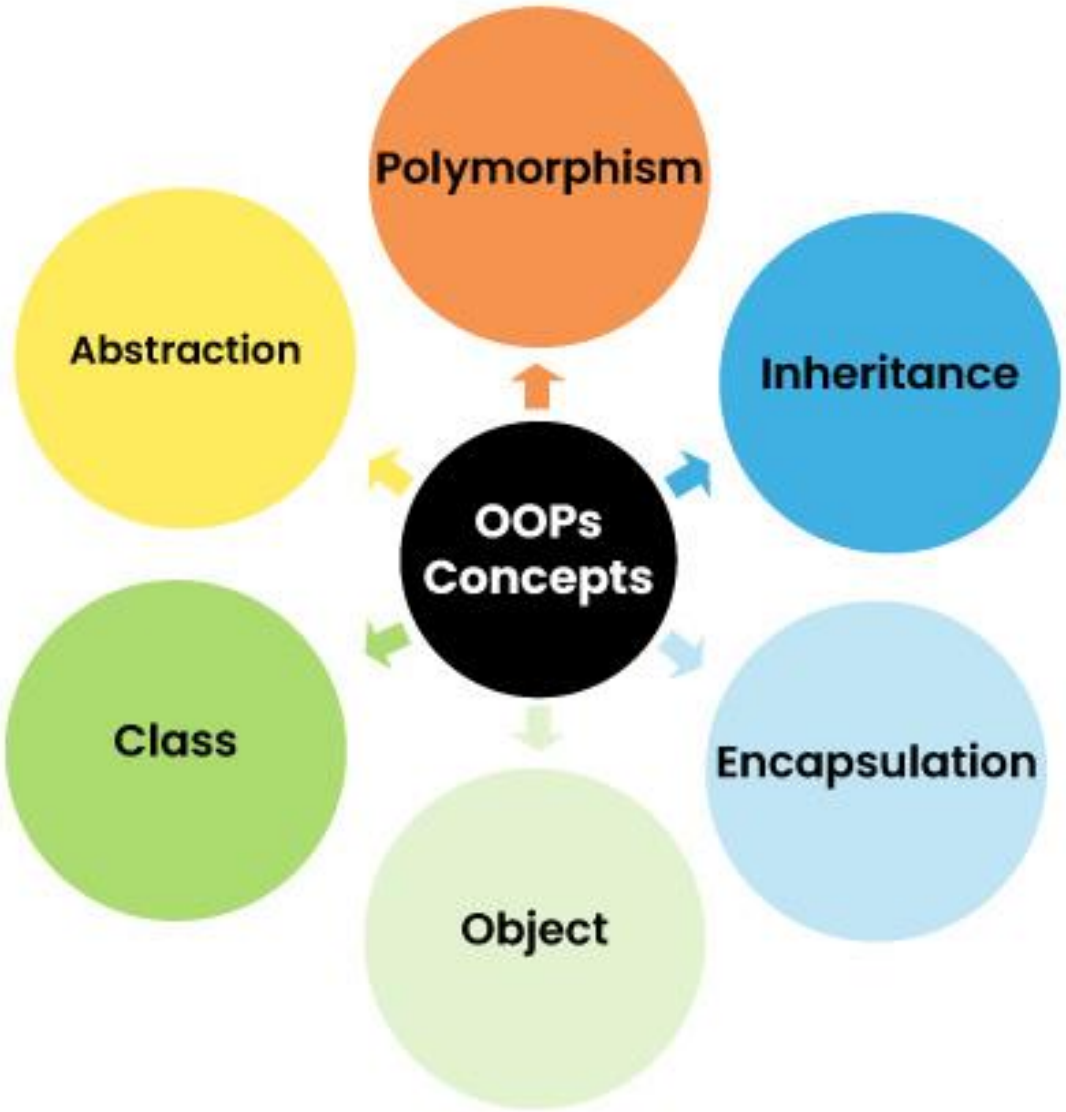
Early Foundations

- **1960s** → The concept of **objects** originated from early programming languages trying to handle complexity.
- The first notable language with object-like ideas was **Simula (1967)**, developed in Norway by Ole-Johan Dahl and Kristen Nygaard.
  - **Simula 67** introduced:
    - Classes
    - Objects
    - Inheritance
    - Virtual methods
  - It was designed for simulations but laid the foundation for OOP.

Key Milestones Recap

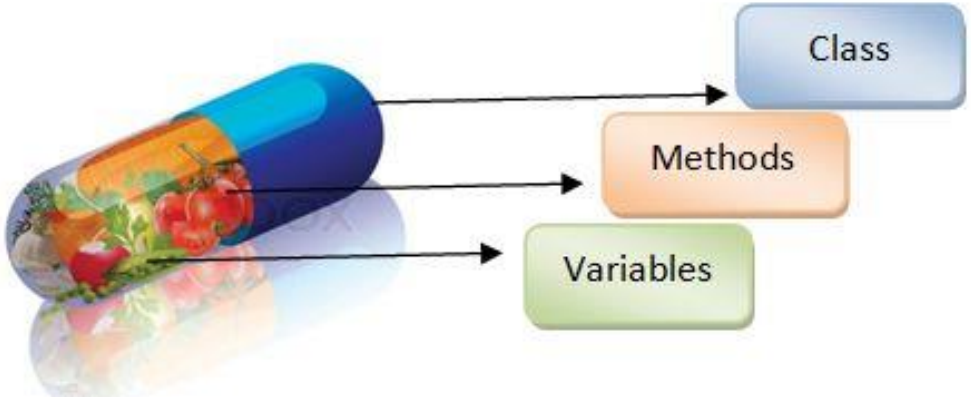
Year	Language	Contribution
1967	Simula	First OOP language, classes & objects
1972–1980s	Smalltalk	Pure OOP, message passing
1985	C++	OOP + low-level systems programming
1995	Java	Safe, portable OOP for the web
2000+	C#, Python, Ruby	Modern OOP, integration with web/devops
2010+	Scala, Kotlin	Blending OOP and functional paradigms

Object-oriented programming Principles



Encapsulation

- **Definition**: Wrapping data (variables) and methods (functions) together as a single unit.
- **Purpose**: Protects internal state by restricting direct access to some of the object's components.



Class

1. A class is a blueprint for the object.
2. The mechanism that allows you to combine data members and the member function in a single unit is called a class.
3. Class is a user-defined data type.
4. Class describes both the properties (data) and behaviors (functions) of objects.
5. Classes are not objects, but they are used to instantiate objects.
6. No memory is allocated at the time of declaration

Class - Example

```
class Car
{
        // Data members (fields)
        String color;
        int speed;

        // Member function (method)
        void drive()
         {
                System.out.println("The car is driving.");
           }
}
```

Object

- A class variable is called an object or instance.
- The object is a real-world entity.
- An Object in has two characteristics:
  - State
  - Behavior
- Each object has different data variables, but they share the member functions.
- Sufficient memory space will be allocated for all the variables/objects of the class at the time of declaration.

Syntax:

ClassName objectName = new ClassName();

Object - Example

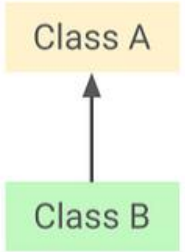
```
public class Main {
public static void main(String[] args)
        {
            Car myCar = new Car(); // Creating an object
            myCar.color = "Red"; // Setting field
            myCar.speed = 100;
            myCar.drive(); // Calling method
            System.out.println("Color: " + myCar.color);
            System.out.println("Speed: " + myCar.speed);
        }
}
```

Java Implementation (Encapsulation):

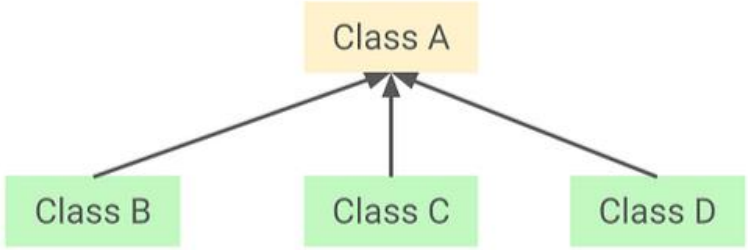
```
class Student
{
        private int age; // private = encapsulated

        public void setAge(int a)
                {
                        if (a > 0)
                        age = a;
                }

                public int getAge()
                {
                                return age;
                }
}
```

Inheritance

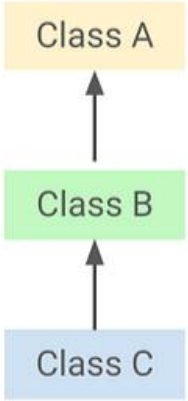
- **Definition**: Allows a class to inherit properties and methods from another class.
- **Purpose**: Promotes code reusability.



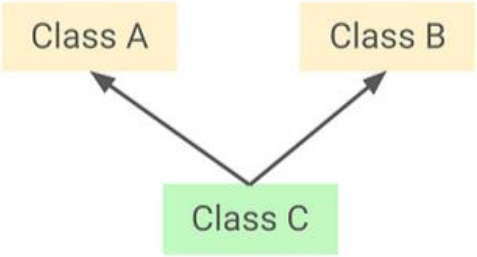
Single Inheritance



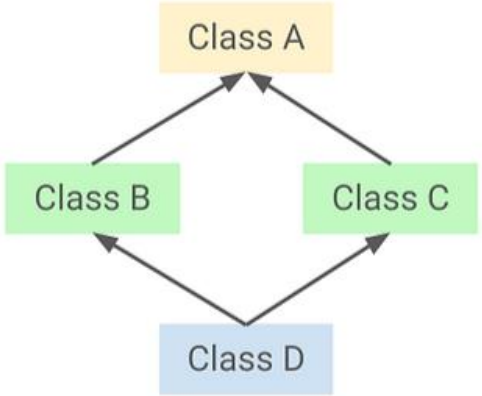
Hierarchical inheritance



Multilevel Inheritance



Multiple Inheritance



Hybrid Inheritance

❌ **Not Supported Directly:** Multiple Inheritance (with classes) - [Interface is required]
Java does NOT support multiple inheritance with classes to avoid ambiguity (diamond problem).

Java Implementation (Inheritance):

```
class Animal
{
        void eat()
        {
                System.out.println("Eating...");
        }
}

class Dog extends Animal
{
        void bark()
        {
                System.out.println("Barking...");
        }
}
```

Dog inherits eat() from Animal.

Polymorphism

- Poly means **many,** and morphs means **forms.** So polymorphism is the ability to take more than one form.
- An operation may exhibit different behaviours in different instances. The behaviour depends upon the type of data used in the operation.

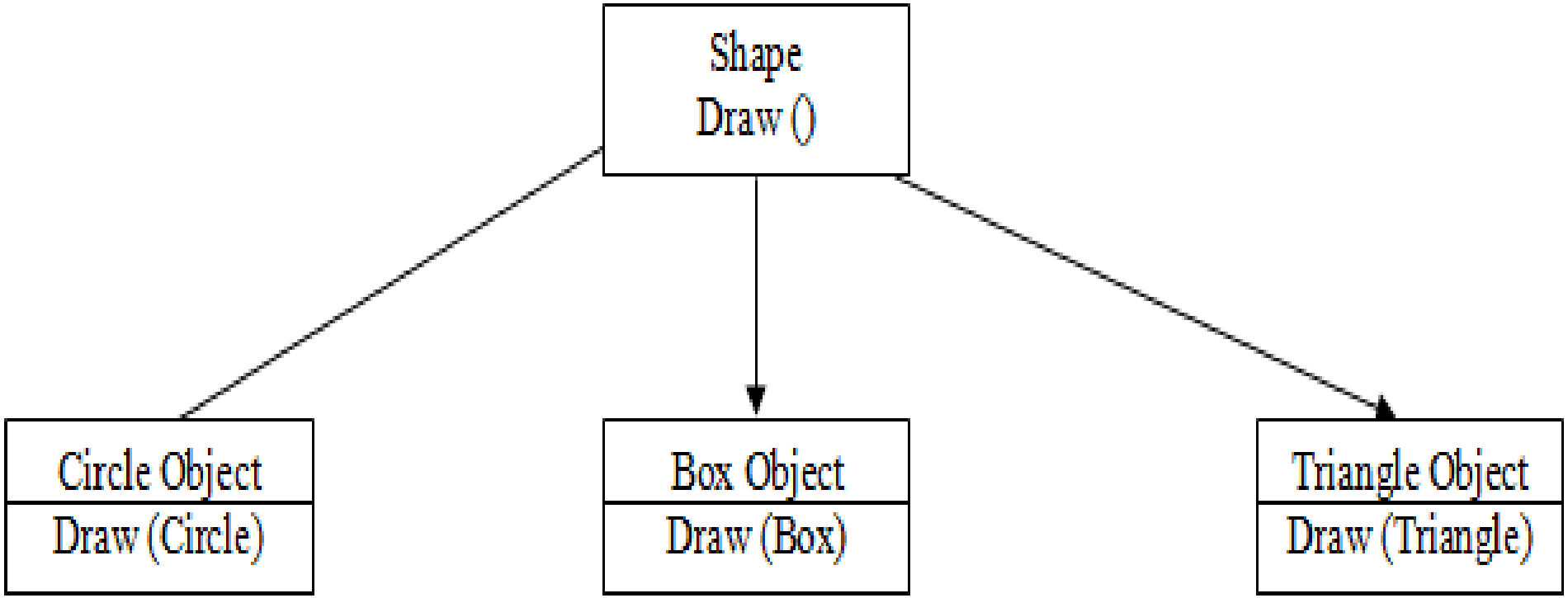
In Shopping malls behave

In Bus behave like Passeng

In School behave like Stud

At Home behave like Son

Polymorphism



Java Implementation (Polymorphism):

```
class Shape {
void draw() {
System.out.println("Drawing Shape");
}
}

class Circle extends Shape {
void draw() {
System.out.println("Drawing Circle");
}
}
class Square extends Shape {
void draw() {
System.out.println("Drawing Square");
}
}
public class Test {
public static void main(String[] args) {
Shape s1 = new Circle();
Shape s2 = new Square();

s1.draw(); // Drawing Circle
s2.draw(); // Drawing Square
}
}
```

Note: Even though both are Shape references, the **correct draw() is chosen at runtime.**

Types of Polymorphism

1. Compile-Time Polymorphism (Static) (Method Overloading)

Same method name, but different parameter list. The compiler decides which method to call based on the method signature.

```
class Calculator {
int add(int a, int b) {
return a + b;
}

double add(double a, double b) {
return a + b;
}

int add(int a, int b, int c) {
return a + b + c;
}
}
```

Types of Polymorphism

2. Run-Time Polymorphism (Method Overriding)

A subclass provides a specific implementation of a method that is already defined in its superclass.

```
class Animal {
void sound() {
System.out.println("Animal makes sound");
}
}

class Dog extends Animal {
void sound() {
System.out.println("Dog barks");
}
}

Animal a = new Dog();
a.sound(); // Output: Dog barks (runtime decision)
```

📌 Even though a is of type Animal, **at runtime**, the Dog version of sound() is called.

Abstraction

- **Definition**: Hiding complex implementation details and showing only the essential features.
- **Achieved via**:
  - Abstract classes
  - Interfaces

How is Java different from C ?

- The major difference is that C is a structure-oriented language, and Java is an object-oriented language with a mechanism to define classes and objects.
- Java does not support an explicit pointer type
- Java does not have a preprocessor, so we can't use #define, #include, and #ifdef statements.
- Java does not include structures, unions, and enum data types.
- Java does not include keywords like goto, sizeof and typedef.
- Java adds labeled break and continue statements.
- Java adds many features required for object-oriented programming.

Continues...

How is Java different from C++ ?

- Java doesn't support pointers to avoid unauthorized access to memory locations.
- Java does not include structures, unions, and enum data types.
- Java does not support operator overloading.
- The preprocessor plays a less important role in C++ and has been eliminated entirely in Java.
- Java does not perform automatic type conversions, which results in a loss of precision.

Continues...

How is Java different from C++ ?

- Java does not support global variables. Every method and variable is declared within a class and forms part of that class.
- Java does not allow default arguments.
- Java does not support inheritance of multiple superclasses by a subclass (i.e., multiple inheritance). This is accomplished by using the 'interface' concept.
- In Java, objects are passed by reference only. In C++, objects may be passed by value or reference

Continues...

How is Java different from C++ ?

New features added in Java:

- Multithreading allows two or more pieces of the same program to execute concurrently.
- C++ has a set of library functions that use a common header file. But Java replaces it with its own set of API classes.
- It adds packages and interfaces.
- Java supports automatic garbage collection.
- break and continue statements have been enhanced in Java to accept labels as targets.
- The use of Unicode characters ensures portability.

Continues...

How is Java different from C++ ?

Features that differ:

- Though C++ and Java support Boolean data type, C++ takes any nonzero value as true and zero as false. True and false in Java are predefined literals that are values for a boolean expression.
- Java has replaced the destructor function with a finalize function.
- C++ supports exception handling that is similar to Java's.
  However, in C++, there is no requirement that a thrown exception be caught.

Features of Java

- Simple
- Object-Oriented
- Platform independent
- Secured
- Robust
- Architecture neutral
- Portable
- Dynamic
- Interpreted
- Multithreaded
- Distributed

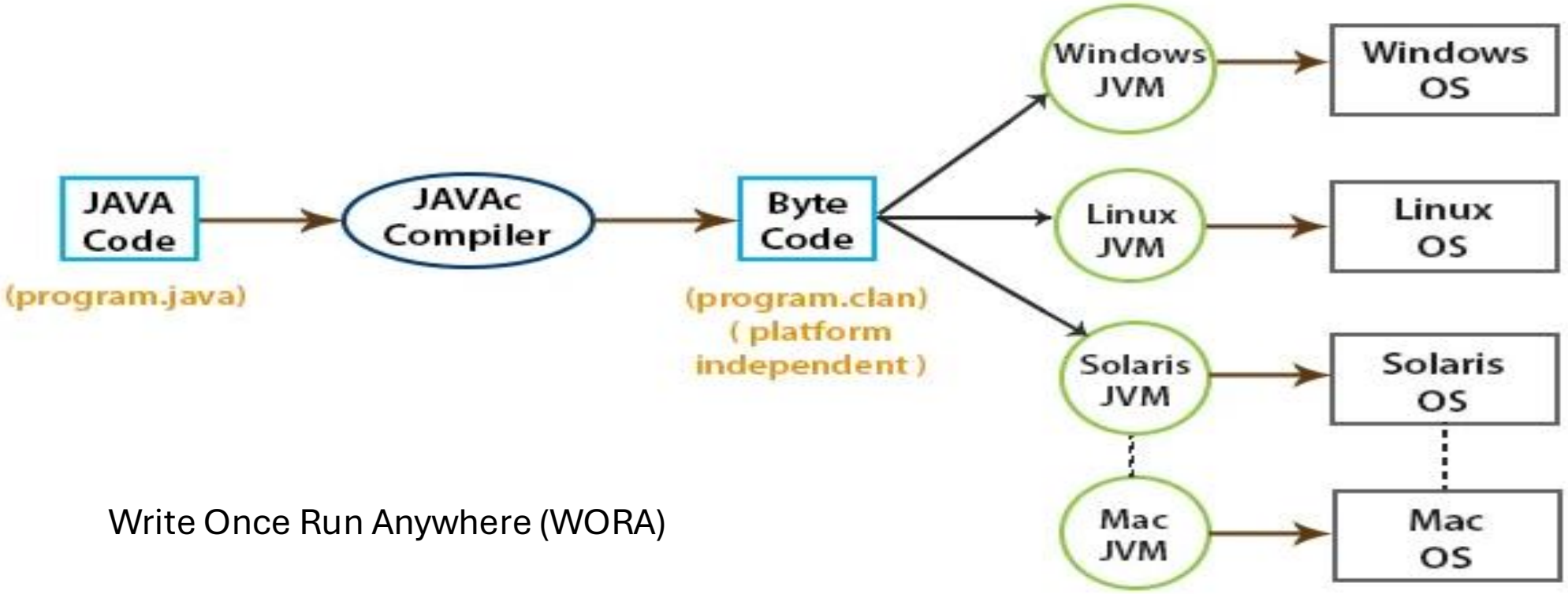
Simple

- According to Sun, Java language is simple because the syntax is based on C++ (so it is easier for programmers to learn it after (++).
- Removed many confusing and rarely-used features e.g., explicit pointers, operator overloading etc.
- No need to remove unreferenced objects because there is an Automatic Garbage Collection in java.

Object-oriented

- Object-oriented means we organize our software as a combination of different types of objects that incorporate both data and behavior.
- The basic concepts of OOPs are:
  - Object
  - Class
  - Inheritance
  - Polymorphism
  - Abstraction
  - Encapsulation

Platform Independent | Portability



Write Once Run Anywhere (WORA)

Secure

- No explicit pointer
- Programs run inside a virtual machine sandbox.
- **Classloader-** adds security by separating the package for the classes of the local file system from those that are imported from network sources.
- **Bytecode Verifier- checks** the code fragments for illegal code that can violate access rights to objects.
- **Security Manager-** determines what resources a class can access, such as reading and writing to the local disk.

Robust

- Robust means strong.
- Java uses strong memory management.
- There is a lack of pointers that avoid security problems.
- There is automatic garbage collection in Java.
- There is an exception handling and type-checking mechanism in Java.

All these points make Java robust.

Interpreted

- You write code in .java file → compiled to .class file (bytecode).
- The **JVM interprets this bytecode**, translating it **line-by-line** into machine code that your computer understands.
- So, Java is **not purely compiled**, and **not purely interpreted** — it's **both**.

Multithreading

Multithreading is a programming and execution model that allows multiple threads to exist within the context of a single process. A thread is the smallest unit of a program's execution, and multithreading enables concurrent execution of multiple threads, allowing them to run independently and perform different tasks simultaneously.

Example: Threads execution in MS-word

Distributed

We can create distributed applications in Java. RMI and EJB are used to create distributed applications. We may access files by calling the methods from any machine on the Internet.

It means **Java supports distributed computing** — you can write programs that **share data, resources, or tasks across different computers** connected in a network.

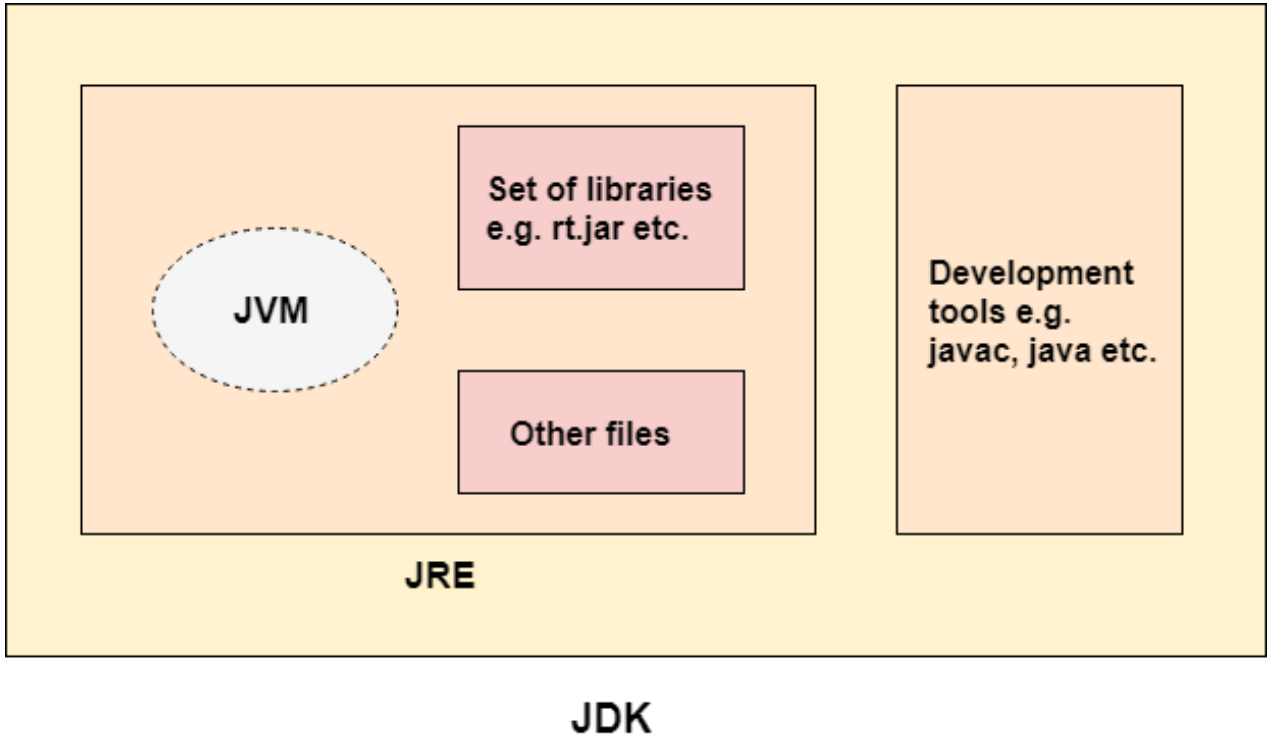
Dynamic

Java is called **dynamic** because it **loads classes, code, and libraries into memory only when they are needed during runtime**, rather than all at once before the program starts.

Real-World Example:

- Think of a **restaurant kitchen** that only cooks a dish when a customer orders it, instead of cooking every possible dish beforehand and letting it sit cold.
- Java only “cooks up” (loads) the parts of the program it actually needs, saving time and memory.

JDK vs JVM vs JRE



Java is a **programming language** and a **platform**.

Java serves as both a programming language, allowing developers to write code, and a platform, providing the tools and runtime environment for executing Java applications across different devices and operating systems.

*Java is **both a language** for building software and a **platform** that guarantees your code will **run the same way** on any device with a JVM — "Write Once, Run Anywhere."*

public static void main(String args[])

- Whether the class contains the main() method or not and whether the main() method is declared according to the requirement or not, these things won't be checked by the compiler. At runtime, JVM is responsible for checking these things.

- At runtime, if JVM is unable to find the required main() method, then we will get a runtime exception

```
class Test
{

}
```

```
javac Test.java
java Test
Error: Main method not found in class Test2, please define the main method as: public static void main(String[] args)
```

public static void main(String args[])

To call by JVM from anywhere

Without an existing object also JVM has to call this main method and the main() method is no where related to any object

main method won't return anything to JVM

This is the name which is configured inside JVM

Command line arguments

Command Line Argurments in C/ C++

int main(int argc, char *argv[])

public static void main(String args[])

- The above syntax is very strict; if we perform any change, we will get a runtime exception.

- Even though the above syntax is very strict, the following changes are acceptable.

  1. The order of modifiers is not important that is instead of "public static" we can take "**static public**" also.

  2. We can declare "String[]" in any acceptable form.

     main(String[] args)
     main(String []args)
     main(String args[])

public static void main(String args[])

3. instead of **args**, we can take any valid Java identifier.

4. We can replace String[] with var arg parameter
   main(String[]args) ==> main(String... args)

5. We can declare the main() method with the following modifiers also.
   - **final**
   - **synchronized**
   - **strictfp**

public static void main(String args[])

```
class Test
{
        final static synchronized strictfp public void main(String... xyz)
        {
                System.out.println("void main");
        }
}
```

javac Test.java
java Test
output: void main

Which of the following are valid main method declarations?

1. public static void main(String[] args)

2. public static int main(String[] args)

3. public static void main(String args)

4. public final synchronized strictfp void main(String[] args)

5. public static final synchronized strictfp void main (String[] args)

6. public static void main(String... args)

System.out.println("");

```
class Test
{
        static String s = “java”;
}

Test.s.length()
```

- **Test** is a class name.
- **‘s’** is a static variable present in the **Test** class of type **String**.
- **length()** is a method present in the **String** class

System.out.println("");

```
class System
{
        static PrintStream out ;
}

System.out.println("Hello");
```

- **System** is a class present in **java.lang** package
- **'out'** is a static variable present in the System class of type **PrintStream**
- **Printin()** is a method present in the **PrintStream** class.

Data Types (Primitive)

Data Type	Size	Default Value	Example
byte	1 byte	0	byte b = 100;
short	2 bytes	0	short s = 5000;
int	4 bytes	0	int x = 100000;
long	8 bytes	0L	long l = 15000000000L;
float	4 bytes	0.0f	float f = 5.75f;
double	8 bytes	0.0d	double d = 19.99;
char	2 bytes	'\u0000'	char c = 'A';
boolean	1 bit (virtually)	false	boolean isTrue = true;

Data Types (Non-primitive)

Data Type	Description	Example
String	Sequence of characters (class)	String name = "Java";
Array	Collection of similar data types	int[] nums = {1, 2, 3};
Class	Blueprint of objects	class Student { ... }
Object	Root class of all Java classes	Object obj = new Student();
Interface	Abstract type (method declarations)	interface Drawable { void draw(); }

Variables

Variable Type	Scope / Lifetime	Description	Example Declaration
◆ **Local Variable**	Inside a method/block only	Declared inside methods or blocks; lifetime ends after method execution	int count = 10; inside main()or any method
◆ **Instance Variable**	Belongs to an object	Declared inside a class but outside methods; each object has its own copy	String name; declared in a class
◆ **Static Variable**	Belongs to the class	Declared with static keyword inside a class; shared by all instances	static int totalStudents;
◆ **Final Variable**	Constant value	Declared with final keyword; value can’t change after initialization	final int MAX_AGE = 100;

Operators

Operator Category	Operators	Description	Example
Arithmetic	+, -, *, /, %	Perform basic math operations	int sum = a + b;
Relational	==, !=, >, <, >=, <=	Compare two values, result is boolean	if (a > b) {...}
Logical	&&, `		, !`
Assignment	=, +=, -=, *=, /=, %=	Assign or update values	x += 5; (same as x = x + 5;)
Unary	++, --, +, -, !	Increment, decrement, plus, minus, NOT	i++;, --j;, !flag
Bitwise	&, `	, ^, ~, <<, >>, >>>`	Operate on bits of integer values
Ternary	? :	Conditional operator (shorthand for if-else)	int max = (a > b) ? a : b;
Instanceof	instanceof	Tests if object is instance of a class/interface	if (obj instanceof String) {...}

Program Control Statements in Java

Type	Statement	Purpose
Conditional	if, if-else, switch	Decision-making
Looping	while, do-while, for	Repetition of code
Jumping	break, continue, return	Transfer control flow

Array 1-D

Syntax	Description
`int[] a = new int[5];`	Declared with fixed size
`int[] a = {1, 2, 3};`	Directly initialized
`int[] a = new int[] {1, 2, 3};`	Explicit with new
`int[] a; a = new int[] {1, 2};`	Separate declaration & initialization
`new int[] {1, 2};`	Anonymous array

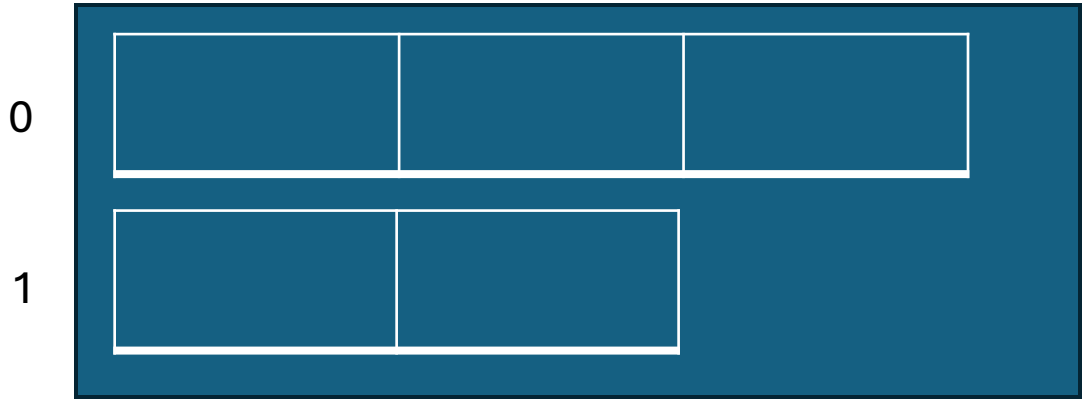
Array 2-D

Syntax	Description
`int[][] a = new int[2][3];`	Fixed-size 2D array
`int[][] a = {{1,2},{3,4}};`	Inline initialization
`int[][] a = new int[][] {{1,2},{3,4}};`	Explicit with new
`int[][] a; a = new int[][] {...};`	Separate declaration
`int[][] jag = new int[2][];`	Jagged array

Jagged Array or Irregular Arrays

```
int[][] jagged = new int[2][]; // 2 rows, no column sizes yet
jagged[0] = new int[3]; // row 0 has 3 columns
jagged[1] = new int[2]; // row 1 has 2 columns
```

You must specify **at least the first dimension** during new if not assigning values directly:



Strings in Java

A **string** is a **sequence of characters**. In Java, strings are **immutable** (i.e., once created, they cannot be changed).

Ways to Create Strings

```
String s1 = "Hello";
String s2 = new String("Hello");
```

Example

```
public class StringExample {
    public static void main(String[] args) {
        String name = "Java Programming";
        System.out.println("Length: " + name.length());
        System.out.println("Uppercase: " + name.toUpperCase());
        System.out.println("Substring: " + name.substring(5));
        System.out.println("Char at 0: " + name.charAt(0));
        System.out.println("Replaced: " + name.replace("Java", "Python"));
    }
}
```

Strings Methods

Method	Description
length()	Returns length of the string
charAt(int index)	Returns char at specified index
substring(int start)	Returns substring from index
equals(String s)	Compares strings (case-sensitive)
equalsIgnoreCase()	Compares strings (case-insensitive)
compareTo(String s)	Lexicographically compares two strings
toLowerCase()	Converts to lowercase
toUpperCase()	Converts to uppercase
trim()	Removes whitespace from start and end
replace(old, new)	Replaces characters
split(String regex)	Splits based on a regex/delimiter
concat(String s)	Concatenates strings

== vs equals() vs equalsIgnoreCase() vs compareTo()

1. == (Reference Comparison)

Compares: Memory addresses (references), not values.
Returns: true if both references point to the same object.

2. .equals() (Value Comparison)

Compares: Content of the strings.
Returns: true if the sequences of characters are the same (case-sensitive).

3. .equalsIgnoreCase() (Value Comparison, Case-Insensitive)

Compares: Content of the strings, ignoring case.
Returns: true if the character sequences are the same, ignoring case.

Continue...

== vs equals() vs equalsIgnoreCase() vs compareTo()

4. compareTo() (Lexicographical Comparison)

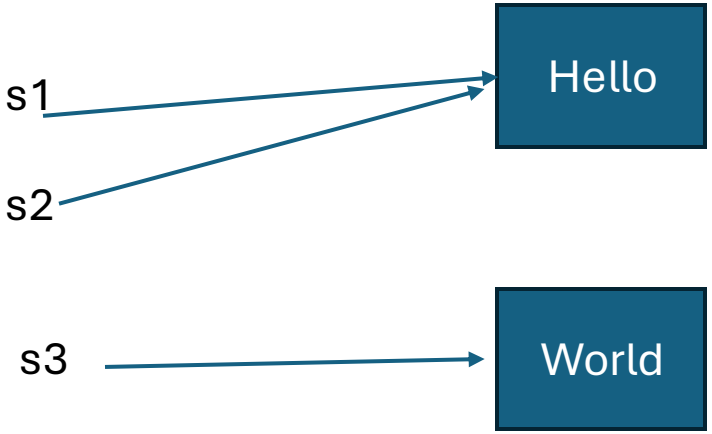
Compares: Unicode values of characters.
Returns:

- 0 if strings are equal
- Negative if the calling string is lexicographically less
- Positive if it's greater

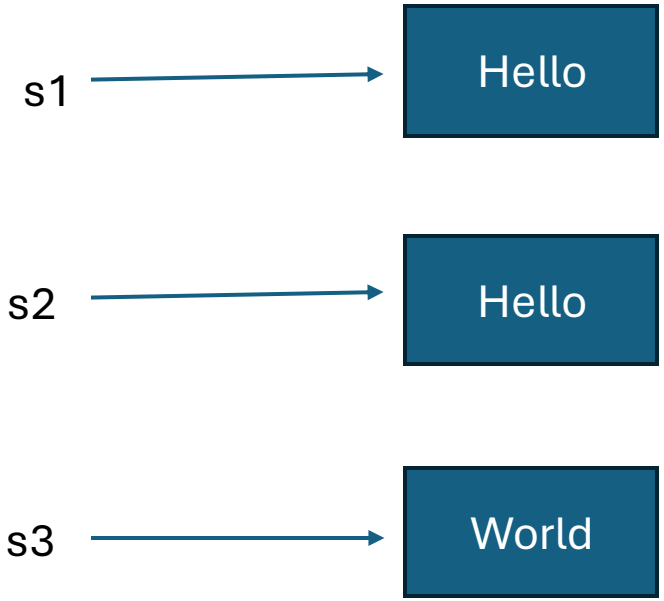
Summary

Method	Type	Case Sensitive	Compares Values	Compares References
==	Reference check	N/A	❌	✅
equals()	Method	✅	✅	❌
equalsIgnoreCase()	Method	❌	✅	❌
compareTo()	Method	✅	✅ (lexicographic)	❌

```
String s1 = “Hello”;
String s2 = “Hello”
String s3 = “World”;
```




```
String s1 = new  String (“Hello”);
String s2 = new  String (“Hello”);
String s3 = new String(“World”);
```



String | StringBuffer

Aspect	String	StringBuffer
Mutability	Immutable – once created, cannot be changed	Mutable – can be modified without creating a new object
Thread Safety	Not thread-safe	Thread-safe (methods are synchronized)
Performance	Slower for repeated modifications (due to new object creation)	Faster for repeated modifications (same object modified)
Memory Usage	Creates a new object on every change	Reuses the same memory space (until capacity is exceeded)
Length	No method like length() – use length() on the object for string length	length() returns current number of characters stored
Capacity	No concept of capacity – memory grows with content	Has an internal buffer with initial capacity (default 16), expands as needed
Expansion Rule	Not applicable	If exceeded, expands using: (oldCapacity × 2) + 2
Usage	Ideal for constants or when the string doesn't change often	Ideal when string needs to be updated multiple times (e.g., in loops, editors)
Package	java.lang.String	java.lang.StringBuffer